OOP

Artem Beznos

Arena Friends: Sword Duel

Zadanie 1 - Špecifikácia

Názov: Arena Friends: Sword Duel

Typ hry: Lokálna 2D arénová bojová hra pre dvoch hráčov na jednom počítači (jedna klávesnica).

O čo v hre ide

Dvaja hráči bojujú proti sebe v uzavretej aréne. Každý hráč ovláda svojho bojovníka, pohybuje sa v 4 smeroch a útočí mečom. Pri zásahu súper stráca zdravie (HP). Počas súboja sa v aréne náhodne objavujú bonusy, ktoré krátkodobo menia priebeh boja – napríklad dočasne zvýšia rýchlosť alebo poškodenie, prípadne okamžite doplnia zdravie. Arény obsahujú steny (prekážky) a spomaľovacie zóny (mláky), ktoré ovplyvňujú pohyb a nútia hráčov taktizovať. Počas boja je na obrazovke zobrazený HUD s informáciami o HP, skóre, aktívnych dočasných efektoch (zostávajúci čas) a o cooldown-e útoku.

Kedy hráč vyhrá

Zápas sa hrá na best-of-3. Kolo vyhrá hráč, ktorý zníži súperovi HP na nulu. Zápas vyhrá hráč, ktorý ako prvý vyhrá 2 kolá.

Herná mechanika (prehľad)

- Pohyb v 4 smeroch
- Útok mečom s cooldownom 1.0 s (proti spamovaniu)
- Každý hráč má 100 HP
- Zásah mečom uberá 10 HP
- Výber mapy v hlavnom menu (Mapa 1 / Mapa 2)
- Bonusy (power-upy) sa objavujú náhodne každých 10–15 s:
    - Liečenie (okamžité doplnenie HP)
    - Dočasné zrýchlenie (5 s)
    - Dočasné zvýšenie poškodenia (5 s)
- Mláky spomaľujú pohyb hráča, kým sa v nich nachádza
- HUD zobrazuje: HP, skóre kôl, zostávajúci čas dočasných efektov a zostávajúci čas cooldown-u útoku

Postavy

Dvaja bojovníci (Player 1 a Player 2) s rovnakými vlastnosťami; líšia sa iba ovládaním.

Mapy (levely)

- Mapa 1 – aréna so stenami a mlákami

- Mapa 2 – aréna s mlákami
- Mapa 3 – aréna so stenami a mlákami

Objekty v hre

- hráči
- meč / zásahová zóna útoku
- bonusy (liečenie, zrýchlenie, zvýšenie poškodenia)
- steny / prekážky
- spomaľovacie mláky
- HUD (Heads-Up Display): HP, skóre, zostávajúci čas dočasných efektov, cooldown meča

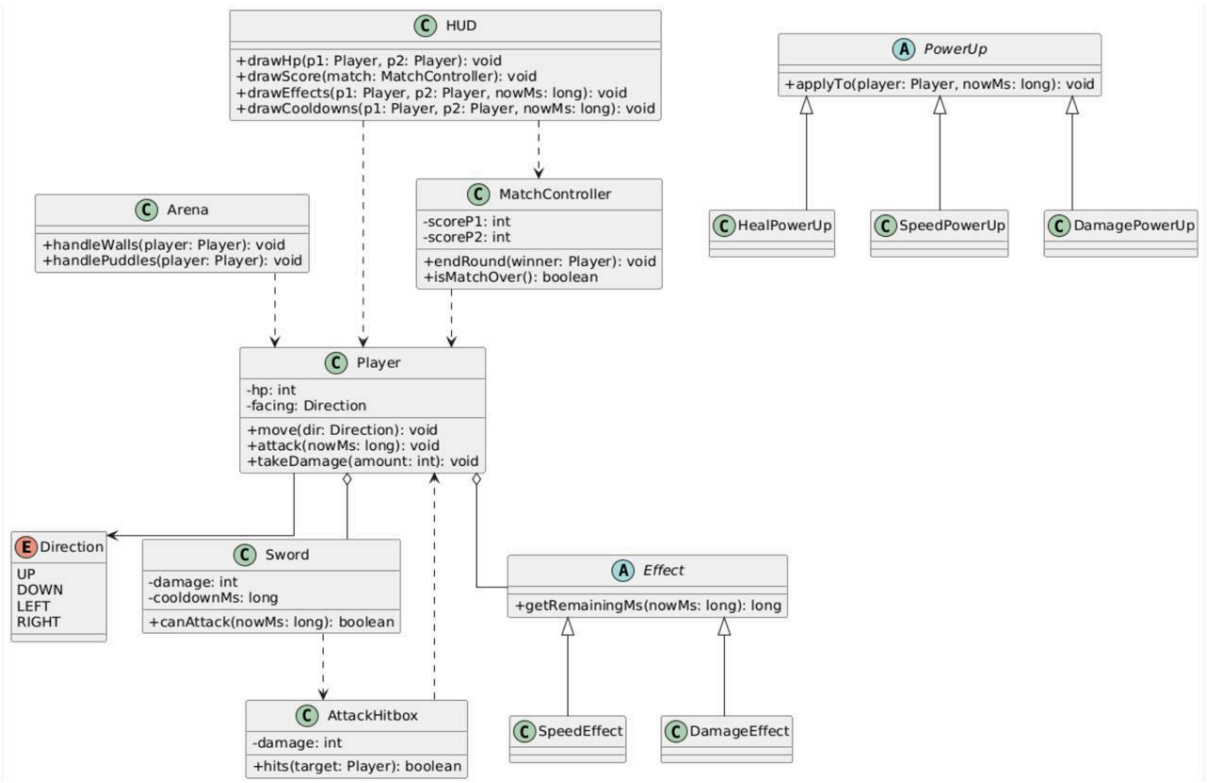
2. Funkcionálne požiadavky

- P0
  - 1. Menu zobrazí hlavné menu s možnosťami Start a Exit.
  - 2. Menu umožní hráčovi vybrať mapu (Mapa 1, Mapa 2 alebo Mapa 3) pred štartom hry.
  - 3. Hra spustí zápas po zvolení mapy.
  - 4. Aréna načíta zvolenú mapu a zobrazí arénu v jednom okne.
  - 5. Aréna zabezpečí hranice mapy, aby hráči nemohli odísť mimo mapy.
  - 6. Ovládanie umožní ovládanie Player 1 pomocou WASD a útok cez kláves Space.
  - 7. Ovládanie umožní ovládanie Player 2 pomocou šípok a útok cez kláves Shift.
  - 8. Hráč aktualizuje smer „facing“ podľa posledného smeru pohybu.
  - 9. Boj nastaví hráčom 100 HP na začiatku kola.
  - 10. Boj vykoná útok do smeru „facing“ ako zásahovú zónu pred hráčom.
  - 11. Boj pri zásahu mečom odoberie súperovi 10 HP.
  - 12. Boj zabezpečí cooldown útoku 1.0 s pre každého hráča.
  - 13. Kolo ukončí kolo, ak HP hráča klesne na 0.
  - 14. Zápas realizuje pravidlá best-of-3 (víťaz je hráč s 2 vyhratými kolami).
  - 15. Zápas po ukončení kola pripočíta bod víťazovi kola a resetuje pozície hráčov.
  - 16. HUD zobrazí HP oboch hráčov počas hry.
  - 17. HUD zobrazí skóre kôl a číslo aktuálneho kola.
- P1
  - 18. Aréna implementuje prekážky (steny) s kolíziou – hráč cez stenu neprejde.
  - 19. Aréna implementuje spomaľovacie zóny (mláky), ktoré znížia rýchlosť hráča počas pobytu v zóne.
  - 20. Bonusy generujú (spawn) bonusy náhodne každých 10–15 s, max. 1–2 bonusy naraz.
  - 21. Bonusy po zdvihnutí liečenia okamžite doplnia HP (bez prekročenia 100).

- 22. Bonusy po zdvihnutí zrýchlenia aplikujú dočasný efekt zvýšenej rýchlosti na 5 s.
- 23. Bonusy po zdvihnutí zvýšenia poškodenia aplikujú dočasný efekt zvýšeného damage na 5 s.
- 24. HUD zobrazí aktívne dočasné efekty a zostávajúci čas ich trvania.
- 25. HUD zobrazí zostávajúci čas cooldown-u útoku pre oboch hráčov.

- P2
  - 26. UI zobrazí obrazovku Game Over po skončení zápasu s možnosťami Restart a Back to menu.
  - 27. UI zobrazí krátku hlášku/obrazovku medzi kolami („Round won by Player X“).
  - 28. Boj zobrazí jednoduchú vizuálnu indikáciu útoku (krátka animácia meča).
  - 29. Hra umožní pauzu a pokračovanie (Pause/Resume) počas hry.

3. UML diagram tried



4. Nákres UI

Zadanie 2 - Checkpoint

JUnit testy

1) playerTakeDamageShouldNotGoBelowZero

Čo testuje: že HP hráča po zranení neklesne pod 0.
Postup: aplikujeme veľké poškodenie (takeDamage(999)).
Očakávanie: hp == 0 a isDead() == true.
Dôvod: aby nevznikali záporné hodnoty HP a logika smrti fungovala korektne.

2) playerHealShouldNotExceedMaxHp

Čo testuje: že liečenie neprekročí maximálne HP (maxHp).
Postup: znížime HP a viackrát voláme heal().
Očakávanie: hp == maxHp.
Dôvod: aby sa HP nepreliečilo nad maximum (napr. 120/100).

3) arenaHandlePuddlesInPuddleShouldSlowDown

Čo testuje: že hráč v kaluži dostane spomalenie (puddle effect).
Postup: vytvoríme PuddlesOnlyArena(..., false) bez textúr, umiestnime hráča do oblasti kaluže a zavoláme handlePuddles(player).
Očakávanie: speedMultiplier == 0.5.
Dôvod: aby efekt kaluží (spomalenie) fungoval konzistentne.

4) arenaHandleWallsOverlappingWallShouldPushOut

Čo testuje: že kolízia so stenou hráča vytlačí mimo steny.
Postup: vytvoríme WallsOnlyArena(..., false), nastavíme hráča tak, aby sa prekrýval so stenou, zavoláme handleWalls(player).
Očakávanie: hráčov Rectangle sa po spracovaní neprekrýva so stenou (overlaps == false).
Dôvod: aby hráč neprechádzal cez steny a nezasekával sa v kolíziách.

Deklarácia použitých OOP princípov

- **Generalizácia**

```
public interface Arena {  4 usages  4 implementations  kixxippi
    // handle collisions with walls and adjust player position
    void handleWalls(Player player);  3 usages  1 implementation  kixx
    // handle puddle effects
    void handlePuddles(Player player);  3 usages  1 implementation  k
}
```

- **Agregácia**

```
public class GameScreen extends ScreenAdapter {  3

    private Player p1;  12 usages
    private Player p2;  12 usages
```

```
p1 = new Player( x: 80,  y: 80,  w: 64,  h: 64,
        textureRightPath: "players/player1_right.png",
        textureLeftPath: "players/player1_left.png",
        sword1,  numberPlayer: 1);

p2 = new Player( x: virtualWidth - 140,  y: virtualHeight - 140,  w: 64,  h: 64,
        textureRightPath: "players/player2_right.png",
        textureLeftPath: "players/player2_left.png",
        sword2,  numberPlayer: 2);
```

- **Kompozícia**

```
public class WallsOnlyArena extends BaseRectangleArena {  10 usages

    private final Array<WallVisual> wallVisuals = new Array<>();  9 u
```

```
wallVisuals.add(new WallVisual(bottom, loadTextures ? "walls/wall_bottom.png" : null));
```

WallsOnlyArena obsahuje zoznam wallVisuals a sama ich vytvára

- **Dedenie (dve hierarchie dedenia)**

1)
```
public class DamageEffect extends Effect
```

2)
```
public abstract class BaseRectangleArena implements Arena
```

```
public class PuddlesOnlyArena extends BaseRectangleArena
```

```
public class WallsOnlyArena extends BaseRectangleArena
```

```
public class MixedArena extends BaseRectangleArena
```

- **Polymorfizmus (compile-time aj run-time)**
  compile-time

```
//create AttackHitbox or null if on cooldown
public AttackHitbox attack(long nowMs) {  1 usage  kixxippi
    if (sword == null) return null;
    if (!sword.canAttack(nowMs)) return null;

    sword.recordAttack(nowMs);
    Rectangle hitRect = createAttackRect();
    return new AttackHitbox(hitRect, sword.getDamage());
}

// basic attack without bonus damage
public void attack(Player target) {  2 usages  kixxippi *
    attack(target, bonusDamage);
}

// overloaded attack with bonus damage
public void attack(Player target, int bonusDamage) {  1 usage  kixxippi
    long nowMs = System.currentTimeMillis();
    AttackHitbox hitbox = attack(nowMs);
    if (hitbox != null && hitbox.hits(target)) {
        target.takeDamage( amount: hitbox.getDamage() + bonusDamage);
    }
}
```

run-time

```
public class ArenaFactory {  2 usages  kixxippi
    public static Arena createArena(int mapId, float worldWidth, float worldHeight) {  1 usage  kix
        switch (mapId) {
            case 1:
                return new WallsOnlyArena(worldWidth, worldHeight);
            case 2:
                return new PuddlesOnlyArena(worldWidth, worldHeight);
            case 3:
            default:
                return new MixedArena(worldWidth, worldHeight);
        }
    }
}
```

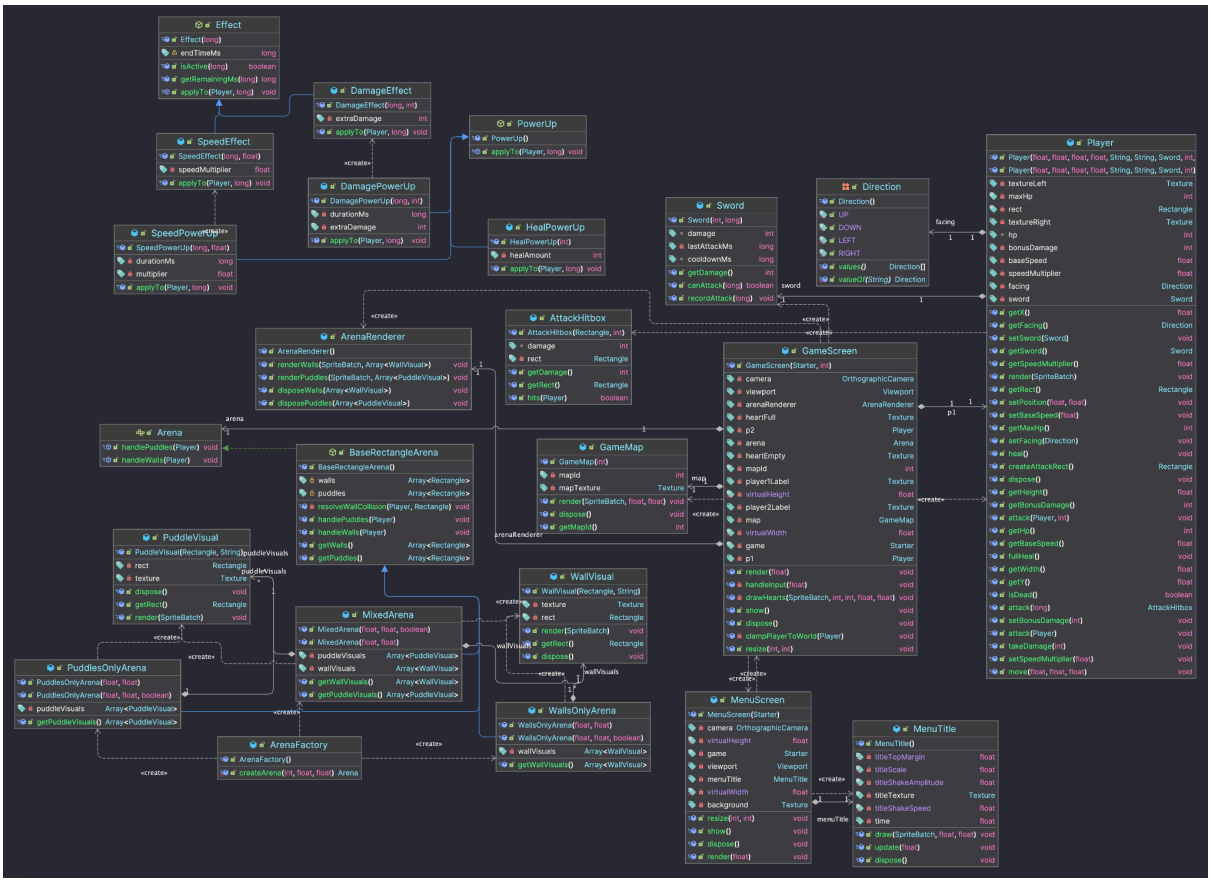
- **Preťaženie (overloading)**

```
//create AttackHitbox or null if on cooldown
public AttackHitbox attack(long nowMs) {  1 usage  kixxippi
    if (sword == null) return null;
    if (!sword.canAttack(nowMs)) return null;

    sword.recordAttack(nowMs);
    Rectangle hitRect = createAttackRect();
    return new AttackHitbox(hitRect, sword.getDamage());
}

// basic attack without bonus damage
public void attack(Player target) {  2 usages  kixxippi *
    attack(target, bonusDamage);
}

// overloaded attack with bonus damage
public void attack(Player target, int bonusDamage) {  1 usage  kixxippi
    long nowMs = System.currentTimeMillis();
    AttackHitbox hitbox = attack(nowMs);
    if (hitbox != null && hitbox.hits(target)) {
        target.takeDamage( amount: hitbox.getDamage() + bonusDamage);
    }
}
```

- **Prekonanie (overriding)**

```java
@Override  2 usages  kixxippi *
public void applyTo(Player player, long nowMs) {
    if (isActive(nowMs)) {
        player.setBonusDamage(extraDamage);
    } else {
        player.setBonusDamage(0);
    }
}
```

Diagram tried



Používateľský manuál – ako spustiť program

Spustenie programu cez terminál

1. Otvor terminál v priečinku projektu (tam kde je súbor build.gradle).

2. Spusti hru:

   Windows: gradlew.bat :lwjgl3:run

   Linux / macOS: ./gradlew :lwjgl3:run

Spustenie cez IntelliJ IDEA

1. Otvor projekt.
2. Vpravo otvor okno Gradle.
3. Spusť úlohu:

   lwjgl3 → Tasks → application → run

Ovládanie (v hre)

Menu:

- *1 – mapa 1 (Walls)*
- *2 – mapa 2 (Puddles)*
- *3 – mapa 3 (Mixed)*
- *ESC – ukončiť / exit*

Hráč 1:

- *Pohyb: W A S D*
- *Útok: SPACE*

Hráč 2:

- *Pohyb: I J K L*
- *Útok: Right ALT/ Right OPTION*